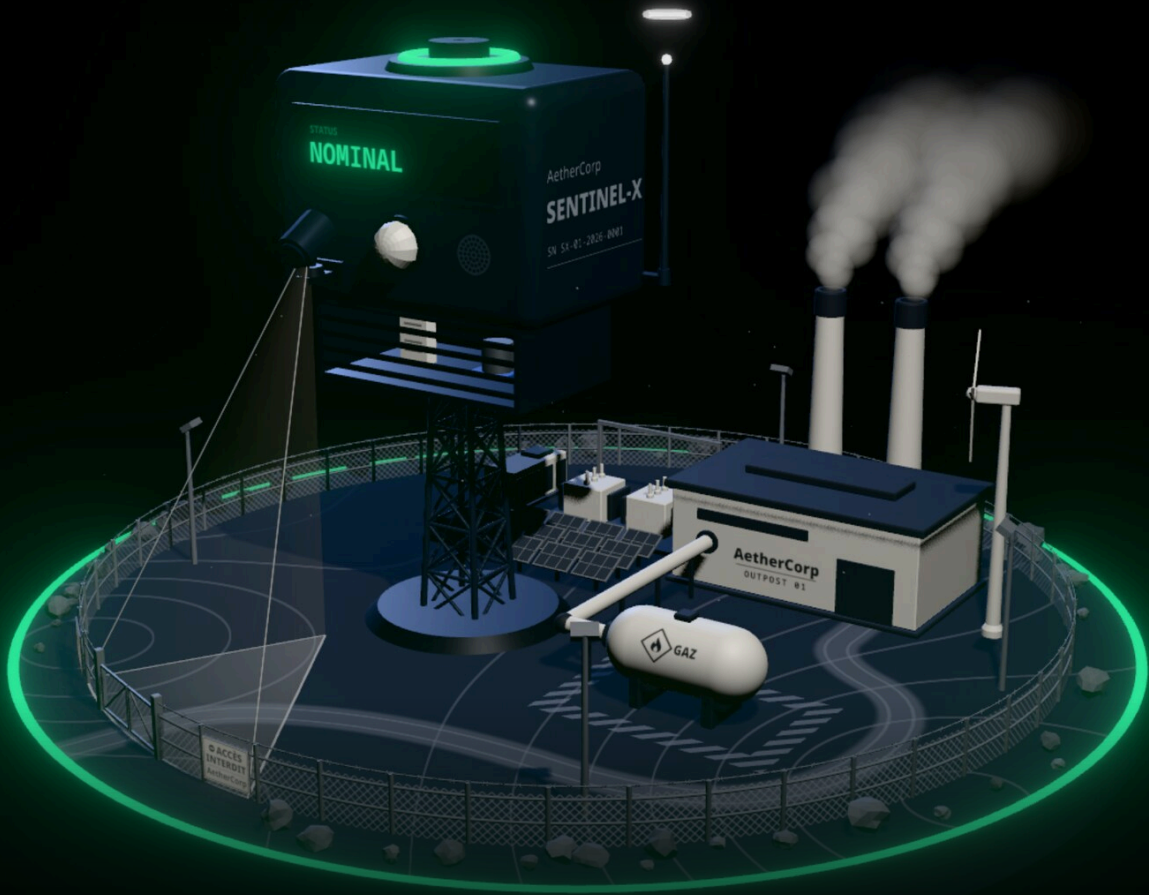




SENTINEL - X

Dossier d'ingénierie technique

Groupe G2 · Option A, centralisation embarquée sur Raspberry Pi

CONTENU

1	Schéma réseau
2	Schéma de câblage électronique
3	Matrice de sécurité
4	Documentation de l'IA
5	Rapport d'audit post - pentest
Annexe	Poster A3

ÉQUIPE

DEV	Arthur · API, jumeau numérique, services IA
DEV	Amen · Dashboard, vue Opérateur
DEV	Nathan · Firmware ESP32, installation du Pi
INFRA	Safi · Infrastructure, réseau, cybersécurité
INFRA	Florent · Infrastructure, réseau, cybersécurité

0 Le système en bref

Sentinel-X protège une micro-centrale isolée contre trois menaces : gaz et surchauffe, intrusion physique, attaque réseau. Un seul boîtier réunit le **Sentinel** (ESP32 et ses sondes) et le **Command Post** (Raspberry Pi 4, le « PC Serveur Local » du sujet). Le PC de l'Opérateur n'apporte qu'un navigateur.

1 s entre deux mesures envoyées par le Sentinel	2 ports de service ouverts à la table : 443 et 8883	TLS 1.2+ sur tout flux qui quitte une machine	6 conteneurs, tous non root et en lecture seule	1 483 tests automatisés, rejoués à chaque pull request
--	---	--	--	--

Réponse au sujet

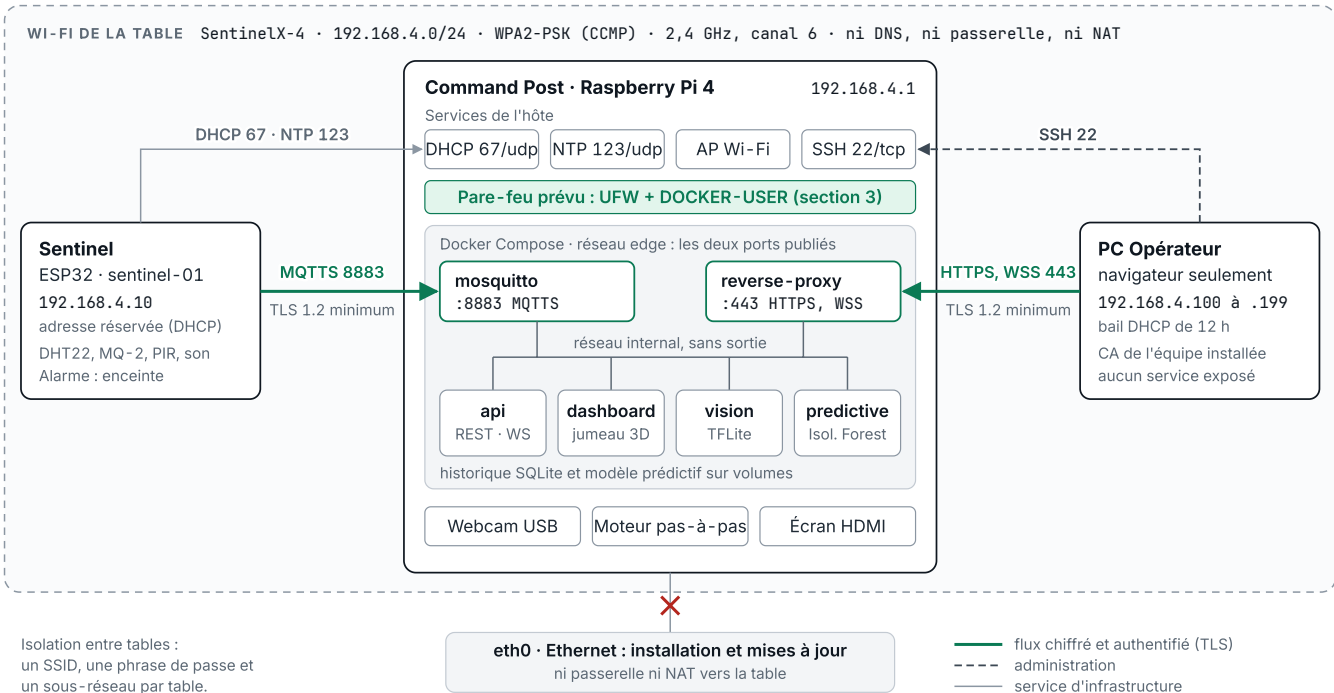
EXIGENCE DU SUJET	RÉPONSE DE L'ÉQUIPE	OÙ
DEV · CONCEPTION ET DÉVELOPPEMENT		
Micrologiciel C++, capteurs cadencés, payloads	Firmware C++ (PlatformIO) : un instantané JSON par seconde, Alertes à hystérésis, Alarme autonome	firmware/
Affichage local du statut	Écran HDMI du Command Post (écart 3)	infra/screen.py
API REST ou WebSocket	API Node.js (Fastify) : REST, WebSocket, POST /api/v1/alerts	backend/
Dashboard : courbes, statut, retour webcam	React : jumeau 3D, courbes, Status, caméra, rejeu d'incident	dashboard/
Commande à distance des actionneurs	Commande de l'Alarme : API, puis MQTTS, puis ESP32	backend/
IA · INTELLIGENCE ARTIFICIELLE ET DATA		
Vision : présence humaine en temps réel	Service vision : EfficientDet-Lite0 (TFLite), classe personne	section 4
Maintenance prédictive sans seuil statique	Service predictive : Isolation Forest, niveaux appris	section 4
Images 640 × 480, moins de 100 ms par trame	Capture en 640 × 480, modèle en 320 × 320, banc de mesure	section 4
INFRA · RÉSEAU ET SYSTÈMES		
Stack Docker-Compose : base, API, broker	6 services, redémarrage automatique, historique SQLite	docker-compose.yml
Point d'accès Wi-Fi dédié, adressage, isolation	Sous-réseau propre à la table, sans route vers l'extérieur	section 1
Monitoring et MCO	Écran de statut, santé Docker, rotation des logs, docker stats	infra/
CYBER · CYBERSÉCURITÉ		
Chiffrement de bout en bout, de l'ESP à la stack	MQTTS (TLS 1.2 minimum), CA de l'équipe ; HTTPS côté Opérateur	section 3
Durcissement du serveur	Script cyber/harden.sh prêt, non appliqué au Pi de démonstration	section 3
Audit de sécurité et pentest croisé	Grille de 19 contrôles, méthode d'audit	section 5

Écarts au sujet

ÉCART	JUSTIFICATION
1 ESP32 à la place de l'ESP8266	Même famille, même chaîne d'outils (Arduino, PlatformIO). Assez de mémoire pour le TLS obligatoire, et plusieurs entrées analogiques contre une seule, limitée à 1 V.
2 Raspberry Pi 4 à la place du Pi 5	Matériel disponible. Inférence plus lente : modèle léger et leviers de la section 4.
3 Écran HDMI sur le Pi à la place de l'OLED sur le microcontrôleur	Même statut IP et Wi-Fi, plus l'état du Sentinel, ses mesures, les Alertes et les conteneurs.

1 Schéma réseau

Le Command Post est le point d'accès d'un sous-réseau Wi-Fi isolé, propre à la table. Rien ne route vers un autre réseau : ni passerelle, ni DNS, ni NAT.



PLAN D'ADRESSAGE · MASQUE 255.255.255.0 · BAIL DHCP DE 12 H

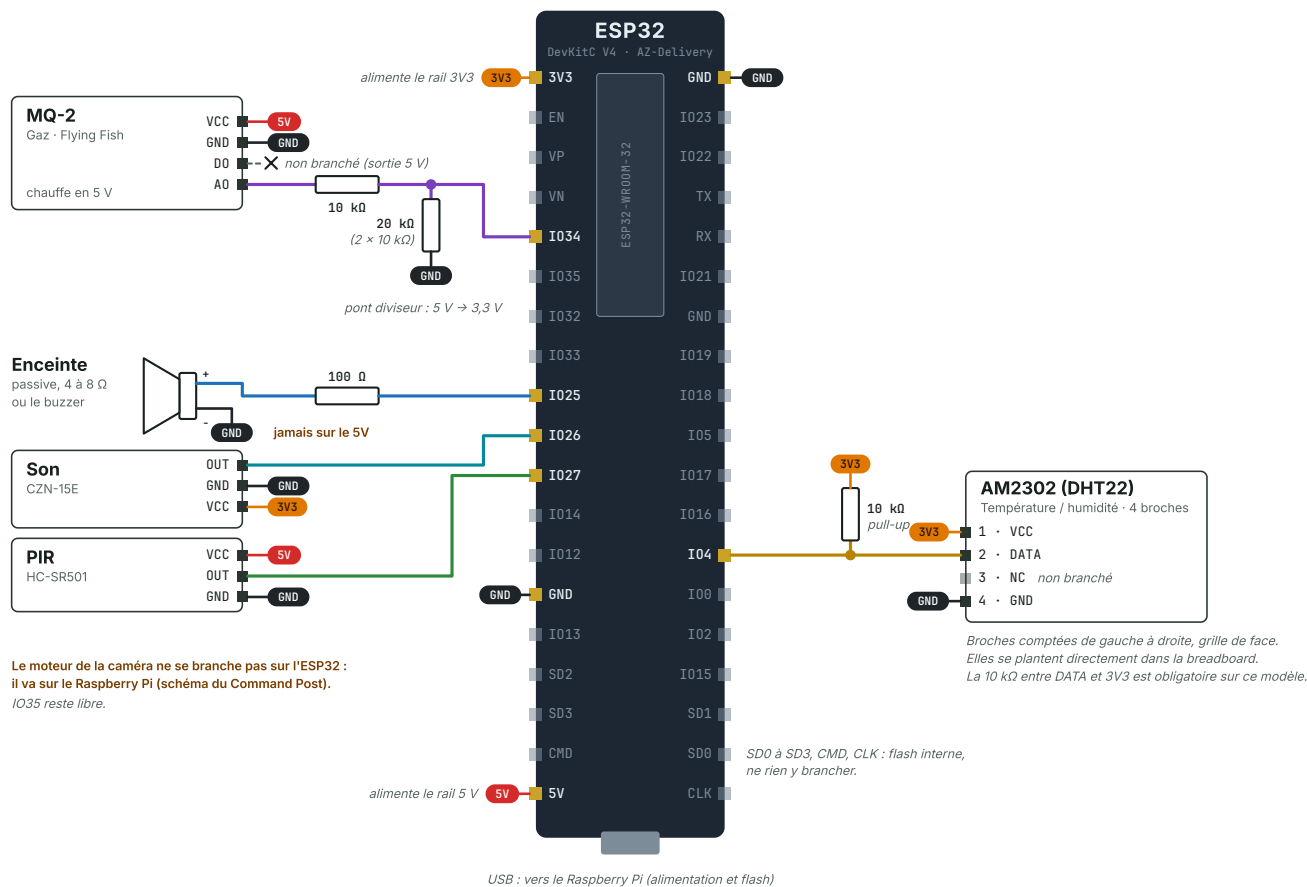
HÔTE	ADRESSE	ATTRIBUTION	RÔLE
Command Post (Raspberry Pi 4)	192.168.4.1	statique, interface wlan0	point d'accès, DHCP, heure, stack Docker
Sentinel (ESP32, sentinel-01)	192.168.4.10	réservée par nom d'hôte	sondes et Alarme
PC Opérateur	192.168.4.100 à .199	DHCP	navigateur, SSH d'administration

MATRICE DES FLUX

DE	VERS	PROTOCOLE, PORT	PROTECTION	CONTENU
Sentinel	mosquitto	MQTT 8883/tcp	TLS 1.2 minimum, CA de l'équipe vérifiée par l'ESP32 ; compte sentinel-01 et ACL	télémétrie et Alertes ; en retour, commandes de l'Alarme
PC Opérateur	reverse-proxy	HTTPS, WSS 443/tcp	TLS 1.2 minimum, CA de l'équipe ; session Opérateur	dashboard, API, temps réel, caméra
PC Opérateur	hôte (sshd)	SSH 22/tcp	compte du Pi ; clé ed25519 seule une fois durci	administration
api, predictive, écran de statut	mosquitto	MQTT 8883/tcp	TLS, CA vérifiée ; un compte et une ACL par client	lecture ; l'api publie les commandes
vision, predictive	api	HTTP 8080, interne	réseau Docker sans sortie, même hôte ; un jeton par service	POST /api/v1/alerts
reverse-proxy	api, dashboard, vision	HTTP, interne	session vérifiée par l'api avant le flux caméra	routage de /api, /ws, /camera
Sentinel, PC Opérateur	hôte	DHCP 67/udp, NTP 123/udp	services d'infrastructure, sans donnée métier	adresses, heure qui date chaque mesure

2 Schéma de câblage électronique

Le Sentinel est un ESP32 DevKitC V4 (38 broches) qui porte les sondes et l'Alarme. La webcam, son moteur et l'écran sont reliés au Raspberry Pi 4. Les broches sont celles de `firmware/include/pins.h`.



Le Sentinel sur breadboard, sans soudure. Fiche de câblage de l'équipe : [docs/cablage/esp32.html](https://docs.cablage/esp32.html).

SENTINEL · ESP32

COMPOSANT	RÔLE	SIGNAL	MONTAGE
DHT22	température, humidité	DATA sur GPIO 4	résistance de tirage de 10 kΩ vers 3V3
MQ-2	gaz, fumées	AO sur GPIO 34 (ADC1)	chauffage en 5 V ; pont diviseur 10 kΩ / 20 kΩ pour que la sortie reste sous 3,3 V
PIR HC-SR501	présence	OUT sur GPIO 27	sortie déjà en 3,3 V
CZN-15E	niveau sonore	DO sur GPIO 26	sortie numérique, seuil réglé au potentiomètre du module
Enceinte 4 à 8 Ω	Alarme (sirène)	GPIO 25	entre la broche et la masse, à travers 100 Ω : environ 30 mA, ce qu'une broche peut fournir ; jamais sur le 5 V

Les sondes analogiques sont sur l'ADC1 : l'ADC2 est indisponible quand le Wi-Fi est actif.

3 Matrice de sécurité

Tout flux qui quitte une machine est chiffré et authentifié. Le durcissement du serveur est scripté, non appliqué au Pi de démonstration. Dernière colonne : tests D1 à D19 de l'audit (section 5).

MESURE	MISE EN ŒUVRE	TEST
CHIFFREMENT DES FLUX (TLS)		
MQTT chiffré seulement	Un seul listener, MQTTS sur 8883, <code>tls_version</code> <code>tlsv1.2</code> ; aucun port 1883	D5, D6, D8
HTTPS seulement	Caddy, TLS 1.2 minimum, en-tête HSTS ; aucun port 80	D5, D7, D8
CA de l'équipe	Autorité de certification ECDSA P-256 créée sur le Pi (<code>infra/setup.sh</code>). Certificats : broker (IP:192.168.4.1, DNS:mosquitto) et proxy (adresse du Pi, <code>sentinel-x.local</code>)	D6, D7
Serveur vérifié par le client	ESP32 : <code>setCACert</code> , jamais <code>setInsecure()</code> . Services et écran de statut : CA imposée. PC Opérateur : CA installée par <code>infra/operator.sh</code>	D6, étape 5
AUTHENTIFICATION ET AUTORISATIONS		
Wi-Fi de la table	WPA2-PSK, CCMP seulement, phrase de passe aléatoire de 24 caractères ; ni passerelle ni NAT	D18
Broker fermé aux anonymes	<code>allow_anonymous</code> <code>false</code> ; un compte par client (<code>sentinel-01</code> , <code>api</code> , <code>predictive</code> , <code>screen</code>), mots de passe aléatoires de 192 bits, fichier haché	D9
Moindre privilège MQTT	ACL par compte : le Sentinel n'écrit que sa télémétrie et ses Alertes, ne lit que ses commandes	D10
Services IA	Un jeton de 256 bits par service, limité à son type d'Alerte ; l'api fixe elle-même la source	D12, D13
Session Opérateur	Mot de passe de 12 caractères ou plus, haché <code>bcrypt</code> ; cookie <code>HttpOnly</code> ; <code>Secure</code> ; <code>SameSite=Strict</code> de 12 h ; origine vérifiée sur le WebSocket	D11, D19
Entrées non fiables	Chaque corps validé contre le schéma (champs inconnus refusés, 16 Ko au plus) ; l'identité du Sentinel vient du topic, pas du message	D13
DURCISSEMENT DU SERVEUR (HARDENING) · CYBER/HARDEN.SH · NON APPLIQUÉ AU PI DE DÉMONSTRATION		
SSH	Clé <code>ed25519</code> seulement ; mot de passe et <code>root</code> refusés ; un seul compte ; depuis le PC Opérateur seulement	D4
Pare-feu de l'hôte	UFW : tout entrant refusé, sauf sur <code>wlan0</code> 22/tcp (PC Opérateur), 67/udp et 123/udp	D1, D2
Ports publiés par Docker	Docker contourne UFW : la chaîne <code>SENTINEL-X</code> , appelée depuis <code>DOCKER-USER</code> , n'ouvre 443 et 8883 qu'au Wi-Fi de la table, et une unité <code>systemd</code> la remet à chaque démarrage	D3
Services inutiles	<code>bluetooth</code> , <code>hciuart</code> , <code>avahi-daemon</code> , <code>triggerhappy</code> , <code>ModemManager</code> coupés	checklist
ISOLATION DES CONTENEURS DOCKER		
Privilèges	Chaque conteneur : utilisateur non <code>root</code> , lecture seule, <code>cap_drop: ALL, no-new-privileges</code> . Aucun <code>privileged</code> , socket Docker jamais monté	D17
Réseau	<code>api</code> , <code>dashboard</code> , <code>vision</code> , <code>predictive</code> sur un réseau Docker interne sans sortie ; seuls le proxy et le broker publient un port	D1, D17
Périphériques	La webcam est passée à <code>vision</code> par son seul nœud <code>/dev</code> , par le groupe <code>video</code>	D17
DÉNI DE SERVICE ET SECRETS		
Limites de débit	Connexion : 5 par minute et par adresse. Commandes : 2 par seconde. Alertes : 20 par seconde et par jeton. Au-delà : 429	D14, D15
Inondation MQTT	<code>max_packet_size</code> 4096, broker sans persistance, logs en rotation (10 Mo × 3 par conteneur) : la carte SD ne se remplit pas	D16
Secrets hors du dépôt	Générés sur le Pi dans <code>infra/secrets/</code> (ignoré par git, droits 600). La CI refuse tout fichier secret suivi	CI

Le durcissement n'est pas appliqué au Pi de démonstration. Le script réserve SSH à un seul PC : nous avons jugé le risque de nous enfermer dehors trop élevé juste avant la démonstration, et nous ne l'avons pas lancé.

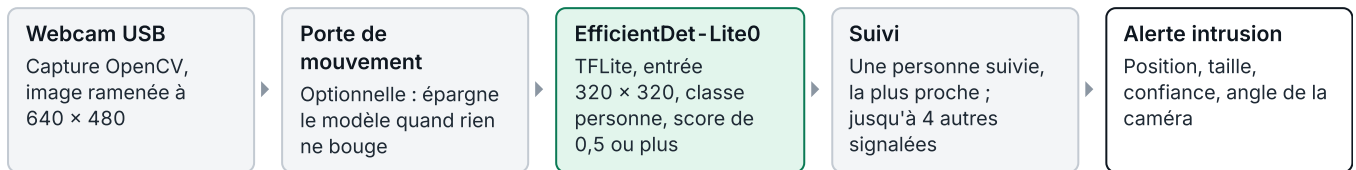
Le script est idempotent. Il a été vérifié hors du Pi, dans un conteneur Debian 12 avec de vrais sshd, UFW et iptables et un réseau de table simulé ; systemd et le vrai Docker restent à vérifier sur le Pi. Il s'applique en 15 minutes et peut être lancé en démonstration : les trois relevés ci-dessous en seront la preuve.

RELEVÉ	COMMANDE	UNE FOIS DURCI	ÉTAT
Ports TCP	<code>nmap -sS -p- 192.168.4.1</code>	22, 443 et 8883 ouverts, les 65 532 autres filtrés	non appliqué
Ports UDP	<code>nmap -sU -p 53,67,68,69,123,137,161,1900,5353 192.168.4.1</code>	123 ouvert, 67 confirmé par dhcp-discover, aucun autre service	non appliqué
Checklist	<code>cyber/harden.sh 4 --check</code>	toutes les lignes cochées, y compris après redémarrage	non appliqué

4 Documentation de l'IA

Deux services Python en conteneur sur le Command Post. Chacun publie ses Alertes par POST /api/v1/alerts avec son propre jeton ; 437 tests automatisés les couvrent, sans matériel.

4.1 Vision : détection d'intrusion service vision · ai/vision/



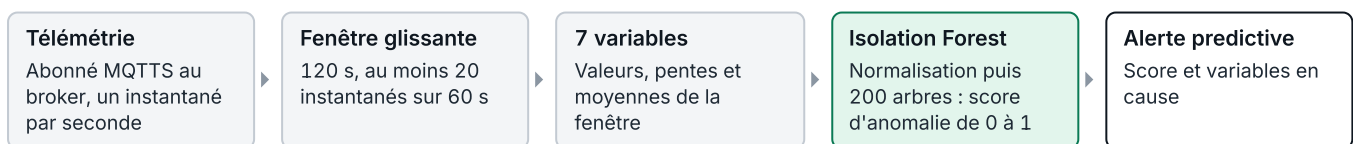
Modèle	EfficientDet-Lite0 (COCO, Apache 2.0), adapté de automaticai/rpi-object-detection (MIT), TensorFlow Lite sur 4 threads. Poids téléchargés à la construction de l'image, sha256 vérifié.
Cycle de l'Alerte	Levée après 2 inférences de suite avec une personne (sévérité critique), mise à jour au plus toutes les 500 ms sous le même identifiant, fin après 3 s sans personne.
Suivi par la caméra	Le moteur pas-à-pas garde la personne suivie au centre de l'image ; sans intrusion, il balaie 30° de chaque côté. L'angle part dans chaque Alerte et le jumeau numérique place l'intrus sur le site.
Retour visuel, repli	Flux MJPEG annoté à 15 images par seconde. Repli autorisé par le sujet : mouvement OpenCV seul (DETECTOR=motion).

PERFORMANCE · EXIGENCE : 640 × 480, MOINS DE 100 MS PAR TRAME · OUTIL : PYTHON -M VISION.BENCH

MACHINE	DÉTECTEUR	INFÉRENCE P50 / P95 (MS PAR IMAGE)
PC de développement (4 vCPU)	TFLite, 4 threads	10,6 / 13,1
	TFLite, 4 threads, porte de mouvement	12,2 / 15,0
	Mouvement seul (repli)	2,1 / 3,0
Raspberry Pi 4	TFLite, 4 threads	à relever
Raspberry Pi 4	TFLite, 4 threads, porte de mouvement	à relever

Leviers si le Pi 4 dépasse 100 ms : une inférence toutes les deux ou trois images (INFER_EVERY), porte de mouvement, repli OpenCV.

4.2 Maintenance prédictive service predictive · ai/predictive/



VARIABLES	SENS	RÈGLE	VALEUR
temp, humidity, air	dernières mesures	Niveaux appris	levée au quantile 0,999, fin au quantile 0,99 des scores d'un nominal tenu à l'écart de l'entraînement
temp_slope, air_slope	tendance par minute (moindres carrés)	Hystérésis	levée après 5 scores de suite au-dessus, fin après 10 de suite en dessous
temp_mean, air_mean	moyennes sur la fenêtre		

Aucun seuil statique. Le modèle apprend le nominal seul, lu dans l'historique de l'api ; les périodes sous Alerte sont écartées, l'entraînement est refusé sous 100 vecteurs.

VALIDATION

JEU DE DONNÉES	CONTENU	VECTEURS	LEVÉE / FIN	RÉSULTAT
Simulation predictive simulate	2 h de nominal synthétique, puis dérive lente chaleur et gaz	7 140	0,602 / 0,580	Alerte 2 min 47 s après le début de la dérive : 43 min 19 s avant le seuil thermique du firmware, 47 min 17 s avant son seuil gaz
Capture réelle (Sentinel)	à relever			

5 Rapport d'audit post-pentest

Notre Command Post se vérifie avec une grille de 19 contrôles, et une table consentante s'audite avec la même méthode.

BASE	Tests automatisés de l'api, rejoués par la CI à chaque pull request, et configuration versionnée du broker, du proxy, de la stack Docker et du Wi-Fi.
OUTILS	Nmap, Wireshark, Metasploit, openssl s_client, mosquito_pub et mosquito_sub, curl
RÈGLES	Tables consentantes du workshop seulement. Ni destruction ni persistance. Dénier de service court et annoncé. Aucun secret conservé.

5.1 Synthèse

19 contrôles : 6 démontrés par les tests automatisés 9 fondés sur la configuration 4 après durcissement

Six contrôles sont démontrés par les tests automatisés de l'api. Neuf se lisent dans les fichiers versionnés du broker, du proxy, de la stack Docker et du Wi-Fi. Les quatre derniers demandent le durcissement du serveur, qui n'est pas appliqué (constat C1).

5.2 Notre Command Post : grille défensive

	CONTRÔLE	COMPORTEMENT	GARANTI PAR	APPUI
D1	Ports TCP exposés (nmap -sS -p-)	22, 443 et 8883 seuls, le reste filtré	docker-compose.yml : seuls le proxy (443) et le broker (8883) publient un port ; le reste est filtré par UFW (harden.sh)	après durcissement
D2	Ports UDP (nmap -sU, scripts dhcp-discover et ntp-info)	67 et 123 seuls	dnsmasq (67) et chrony (123) sur wlan0 ; le reste est filtré par UFW (harden.sh)	après durcissement
D3	Docker ne contourne pas le pare-feu (scan depuis l'Ethernet)	443 et 8883 muets hors du Wi-Fi de la table	chaîne SENTINEL-X appelée depuis DOCKER-USER, remise à chaque démarrage (harden.sh)	après durcissement
D4	SSH par mot de passe	refusé, clé ed25519 seule, depuis le PC Opérateur	sshd : mots de passe et root refusés, clé ed25519, un seul compte (harden.sh)	après durcissement
D5	Ports en clair (MQTT 1883, HTTP 80)	aucun, connexion refusée	mosquitto.conf : un seul listener, 8883 ; Caddyfile : auto_https off, seul le port 443 est publié	configuration
D6	TLS du broker (openssl s_client sur 8883)	TLS 1.2 ou plus, certificat de la CA d'équipe, échec sans la CA	mosquitto.conf : tls_version tlsv1.2 ; certificat du broker signé par la CA (infra/setup.sh)	configuration
D7	TLS du dashboard (openssl s_client sur 443)	HTTPS seul, certificat de la CA d'équipe	Caddyfile : protocols tls1.2, certificat de la CA, en-tête HSTS	configuration
D8	Capture Wireshark d'une session de démo	ni identifiant ni mesure lisible	découle de D5 à D7 : aucun listener en clair sur la table	configuration
D9	Broker sans identifiants	connexion refusée	mosquitto.conf : allow_anonymous false, password_file	configuration
D10	ACL : sentinel-01 publie sur command/...	refusé, chaque compte reste dans son périmètre	infra/mosquitto/acl : sentinel-01 n'écrit que sa télémétrie et ses Alertes	configuration
D11	API, WebSocket et caméra sans session	401	operator-auth.test.ts : 401 sans session, flux temps réel refusé ; /camera derrière forward_auth (Caddyfile)	tests automatisés

	CONTRÔLE	COMPOTEMENT	GARANTI PAR	APPUI
D12	Jeton vision avec un autre type d'Alerte	403	alert-ingress.test.ts : 403 sur un type d'Alerte qui n'est pas celui du jeton	tests automatisés
D13	Alerte au champ sentinelle ou source mensonger	ignoré, valeurs fixées par l'api	alert-ingress.test.ts, mqtt-ingress.test.ts : source tirée du jeton, Sentinel tiré du topic	tests automatisés
D14	Plus de 5 tentatives de connexion par minute	429 avec Retry-After	rate-limit.test.ts : 429 après 5 connexions par minute, même avec le bon mot de passe	tests automatisés
D15	Plus de 2 commandes/s, plus de 20 Alertes/s	rejet au-delà de la limite	rate-limit.test.ts : 429 au-delà de 2 commandes/s et de 20 Alertes/s par jeton	tests automatisés
D16	Message MQTT de plus de 4096 octets, rafale	message rejeté, service maintenu	mosquitto.conf : max_packet_size 4096, persistence false ; logs 10 Mo x 3 (docker-compose.yml)	configuration
D17	Revue des conteneurs	aucun privilège superflu, socket Docker non monté	docker-compose.yml : no-new-privileges, cap_drop ALL, read_only, réseau internal ; aucun privileged	configuration
D18	Revue du point d'accès Wi-Fi	WPA2-PSK, CCMP seul, WPS coupé, pas de NAT	plug-and-play.sh : wpa-psk, rsn, ccmp, phrase de 24 caractères, adressage manuel sans partage ; WPS à contrôler sur le Pi	configuration
D19	Cookie de session et origine du WebSocket	HttpOnly, Secure, SameSite=Strict ; autre origine : 403	operator-auth.test.ts : cookie fermé aux scripts et aux autres sites, page d'une autre origine refusée	tests automatisés

5.3 Méthode d'audit d'une table consentante

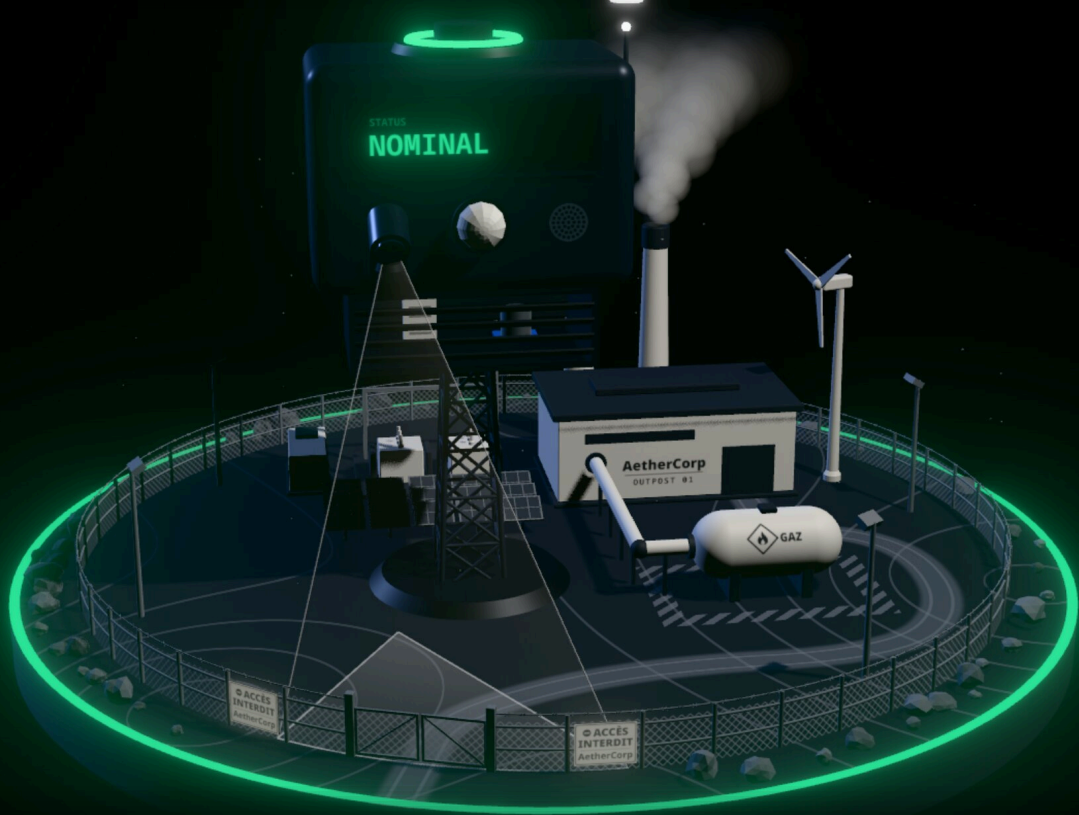
	ÉTAPE	CE QUI EST TENTÉ
1	Reconnaissance (Nmap)	Cartographie des ports TCP et UDP ; services exposés, versions, ports en clair
2	Transport (TLS, Wireshark)	openssl s_client sur chaque port TLS : version, certificat, chaîne de confiance ; capture Wireshark d'une session
3	Authentification (broker, API)	Broker : connexion anonyme, test de l'ACL. API : endpoint protégé appelé sans session
4	Résilience (DoS court, cible prévenue)	Dépassement des limites de connexion, de commandes et d'Alertes ; message MQTT
5	Homme du milieu (avec accord)	Interception et rejeu sur le segment de la table : la vérification de la CA doit les empêcher
6	Restitution à la table auditée	Constats rendus à la table auditée, rien n'est conservé

5.4 Constats et correctifs

	SYSTÈME	CRITICITÉ	CONSTAT	CORRECTIF	ÉTAT
C1	Command Post	Moyenne	cyber/harden.sh n'est pas appliqué : SSH accepte le mot de passe depuis toute la table, le pare-feu d'hôte n'est pas actif, 443 et 8883 répondent aussi par l'Ethernet	cyber/harden.sh 4, 15 minutes (docs/DURCISSEMENT-PI.md)	prêt, non appliqué

5.5 Conclusion

Le chiffrage des flux, l'authentification et les limites de débit sont en place, démontrés par les tests ou lisibles dans la configuration. Il reste à appliquer cyber/harden.sh au Pi (constat C1).



SENTINEL - X

La sécurité à la bordure.

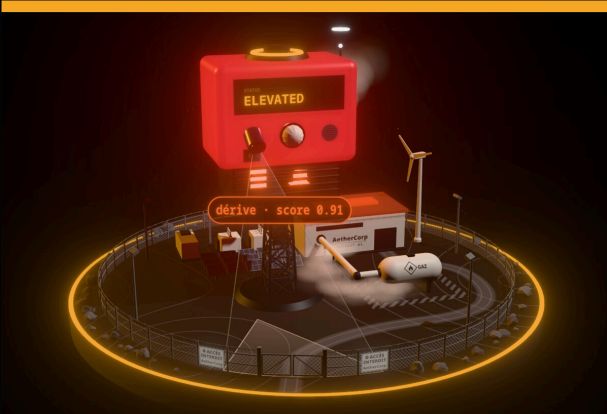
Un boîtier autonome qui surveille, comprend et défend une micro-centrale isolée, là où personne ne peut veiller.



DÉTECTER

Fuite de gaz, surchauffe

Les sondes sont lues chaque seconde. Au seuil critique, la sirène sonne sur place, même serveur coupé.



ANTICIPER

La dérive avant l'incident

Un modèle Isolation Forest apprend le fonctionnement normal et signale une dérive lente bien avant le seuil.



REPÉRER

Intrusion physique

La caméra reconnaît une personne, la suit, et le jumeau numérique 3D la place sur le site en direct.



1 s

entre deux mesures du Sentinel

2

ports de service publiés, tous deux chiffrés

TLS 1.2+

et une autorité de certification propre à l'équipe

7

variables pour le modèle prédictif, aucun seuil fixe

1 483

tests automatisés, rejoués à chaque pull request